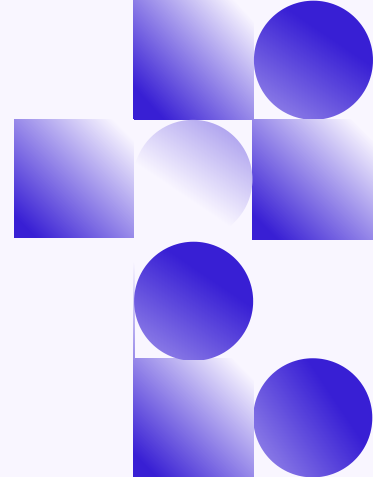


SyncSpace - VITyarthi Project



STUDENT FREE TIME MANAGEMENT & COLLABORATIVE ACTIVITY SCHEDULER

“Find a time when everyone is free.”

PROJECT REPORT

A Java-Based Scheduling System for Students and Collaborative Groups

SUBMITTED BY

Name : Tanisha Sethi

Registration No.: 25BAI11491

Course: BTech (AI & ML)

CORE FUNCTION

Manage student availability → Create groups → Find common free time → Schedule activities → Detect conflicts → Generate reports

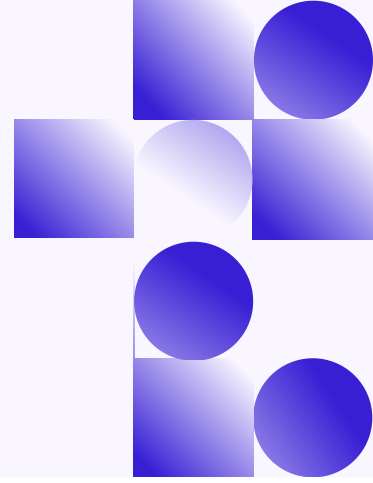
ACADEMIC DETAILS

College : VIT Bhopal University

Faculty : Dr. Sanat Jain

Academic Year : 2026-2027

INTRODUCTION



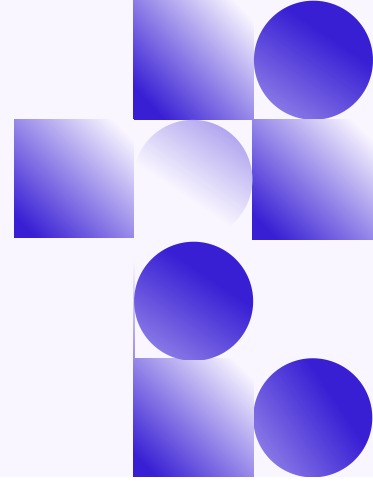
Students often participate in multiple activities such as academic projects, study groups, club meetings, competitions, and other co-curricular activities. Since every student has a different class timetable and personal schedule, finding a common free time for a group can be difficult.

Usually, students coordinate through repeated messages and manually compare their availability. This can take time and may result in scheduling conflicts.

The Student Free Time Management & Collaborative Activity Scheduler is a Java-based application designed to solve this problem. It allows students to maintain their schedules and free-time slots, create or join groups, find common free time among group members, schedule activities, detect conflicts, and view upcoming activities.

The project uses Java programming concepts such as classes and objects, collections, exception handling, file handling, assertions, Path, and Files.

PROBLEM STATEMENT



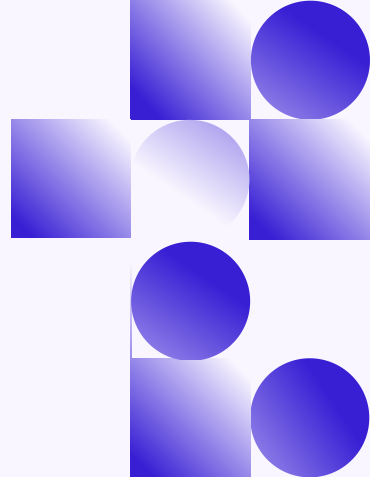
Students belonging to the same project, club, study group, or competition team often have different schedules. Finding a suitable common time for meetings requires manually asking each member about their availability.

This creates several problems:

- Difficulty in finding common free time.
- Repeated communication between members.
- Possibility of scheduling conflicts.
- Wasted time while coordinating.
- Difficulty maintaining records of meetings and activities.
- Lack of a centralized schedule management system.

Therefore, there is a need for a simple system that can store student availability and automatically identify suitable common free-time slots.

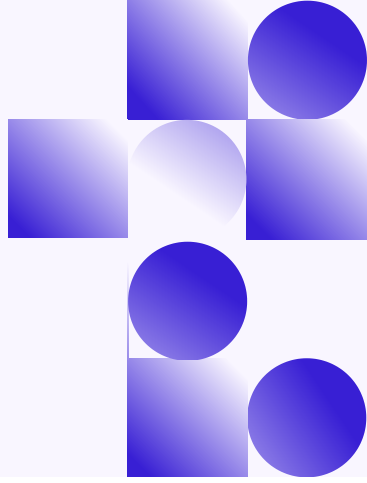
OBJECTIVES



The main objectives of the project are:

1. To maintain student profiles and schedules.
2. To allow students to add and manage their free-time slots.
3. To create and manage groups.
4. To calculate common free time among group members.
5. To schedule project, club, study, and other activities.
6. To detect conflicts between scheduled activities.
7. To display upcoming activities and reminders.
8. To generate basic activity reports.
9. To demonstrate Java programming concepts through a practical application.
10. To provide a simple terminal-based interface that can run without a graphical environment.

SCOPE OF THE PROJECT



The system focuses on managing the availability of students and coordinating group activities.

Included in the project

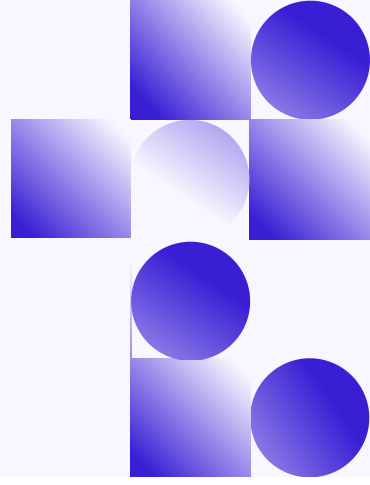
- Student registration
- Student profile management
- Class schedule management
- Free-time management
- Group creation
- Group membership
- Common free-time calculation
- Activity scheduling
- Conflict detection
- Upcoming activity display
- Basic reports
- File-based data storage
- Input validation
- Exception handling

Outside the current scope

- Online video meetings
- Real-time chat
- Mobile application
- University authentication
- Cloud database
- Push notifications

These features can be considered for future development.

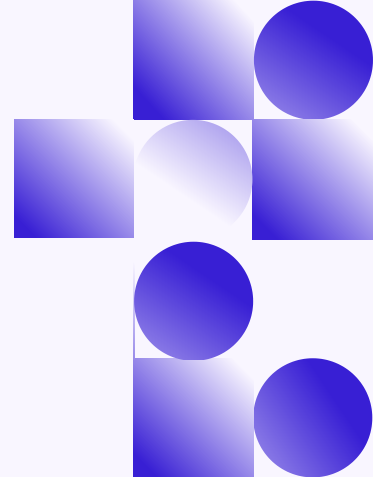
TARGET USERS



The application is mainly intended for:

- College students
- Project teams
- Study groups
- Club members
- Competition teams
- Students involved in co-curricular activities

FUNCTIONAL REQUIREMENTS



1 – Student Management

The system shall allow users to:

- Register a student.
- Enter student ID.
- Enter student name.
- Enter department and year.
- View student information.

2 – Schedule Management

The system shall allow students to:

- Add class schedules.
- Add free-time slots.
- Edit free-time slots.
- Delete free-time slots.
- View their availability.

3 – Group Management

The system shall allow users to:

- Create a group.
- Join an existing group.
- Add members.
- View group members.

Groups may represent:

- Academic projects
- Study groups
- Clubs
- Competitions

- Co-curricular activities

4 – Common Free-Time Finder

This is the **main feature** of the project.

The system compares the free-time slots of all members and finds the overlapping time period.

Example

Student	Free Time
Rahul	4:00 PM – 7:00 PM
Priya	5:00 PM – 8:00 PM
Aman	5:30 PM – 7:00 PM
Neha	6:00 PM – 8:00 PM

The common available time is:

6:00 PM – 7:00 PM

The system displays this as a possible meeting time.

5 – Activity Scheduling

Users can schedule:

- Project discussions
- Study meetings
- Club meetings
- Competition preparation
- Co-curricular activities
- Other group activities

The system stores:

- Activity name
- Activity type
- Date
- Start time
- End time
- Group

- Location

6 – Conflict Detection

Before scheduling an activity, the system checks whether the selected time overlaps with an existing activity.

If a conflict exists, the system displays an appropriate error message.

Example:

ERROR: The selected time overlaps with an existing meeting.

Please choose another time.

7 – Upcoming Activities

The system allows users to view upcoming scheduled activities.

Example:

Upcoming Activities

Project Meeting

Date: 15/09/2026

Time: 5:00 PM - 6:00 PM

Competition Preparation

Date: 17/09/2026

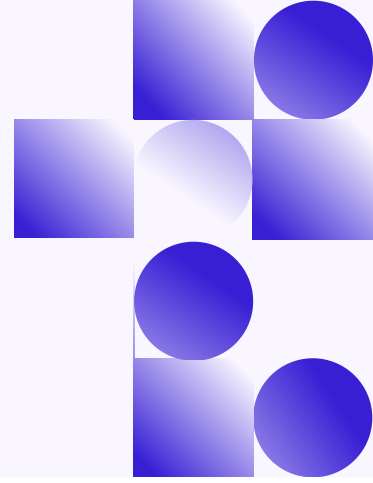
Time: 6:00 PM - 7:30 PM

8 – Reports

The system can generate basic reports such as:

- Total number of activities.
- Number of project meetings.
- Number of club activities.
- Number of conflicts.
- Upcoming activities.
- Activity participation information.

NON FUNCTIONAL REQUIREMENTS



1 – Performance

The common free-time calculation should execute quickly for normal student-group sizes.

2 – Usability

The application should provide a simple menu-driven terminal interface.

3 – Reliability

Invalid operations should not terminate the entire program unexpectedly.

4 – Maintainability

The system should use separate classes and packages for models, services, utilities, and exceptions.

5 – Error Handling

The system should provide meaningful error messages for invalid input, missing data, invalid time slots, and scheduling conflicts.

6 – Security

The application should validate user input and avoid unsafe file operations.

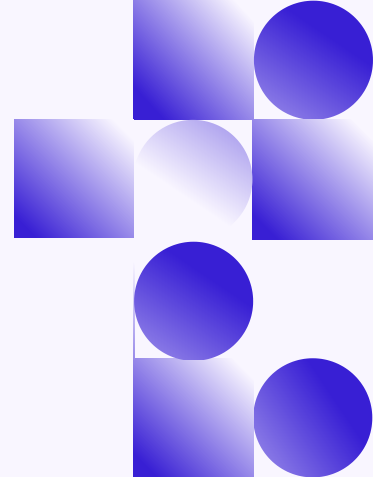
7 – Resource Efficiency

The application should use appropriate Java collections and file operations without unnecessary resource consumption.

8 – Scalability

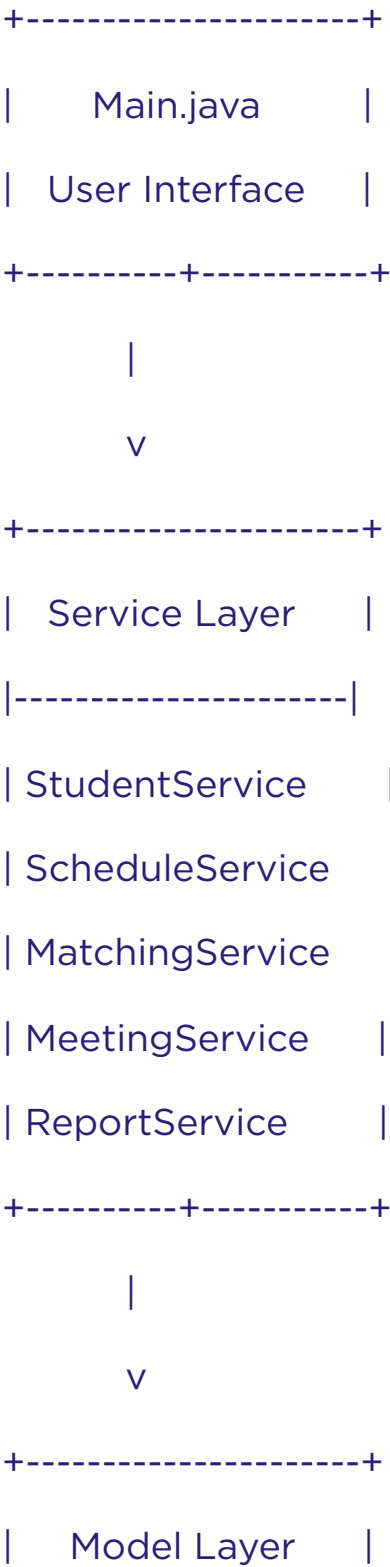
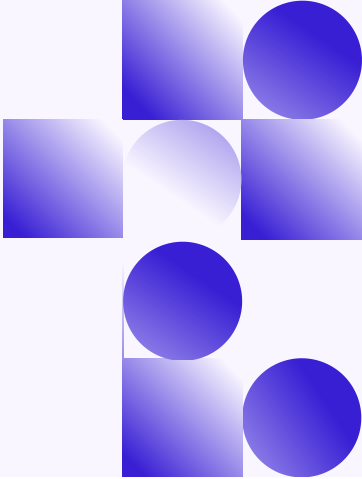
The design should allow additional activities, users, groups, and scheduling features to be added later.

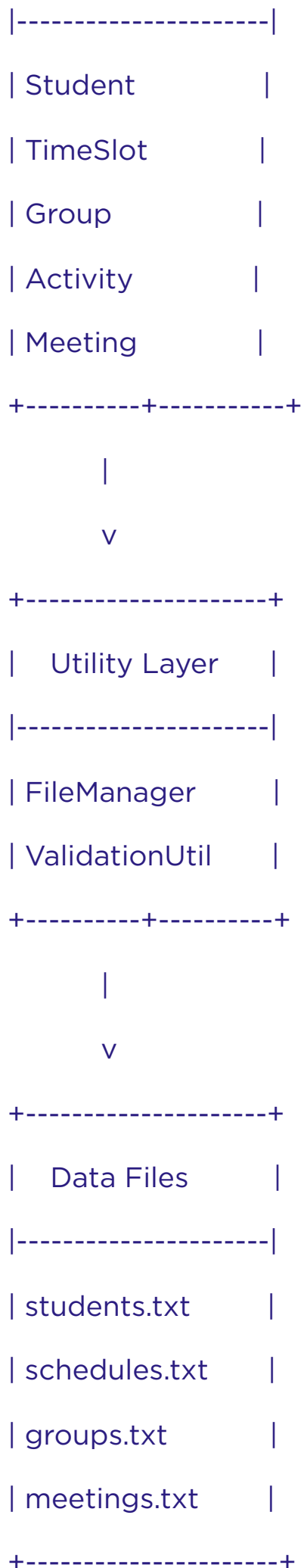
TECHNOLOGY STACK



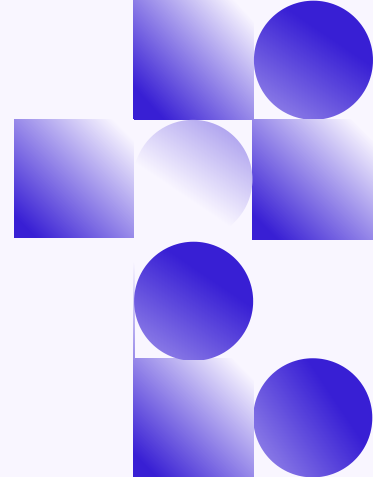
Component	Technology
Programming Language	Java 17
Interface	Command Line / Terminal
Data Storage	Text files
Version Control	Git
Repository	GitHub
IDE	VS Code / IntelliJ IDEA / Eclipse
Build Tool	Maven (optional)
Documentation	Markdown / PDF

SYSTEM ARCHITECTURE





PROJECT MODULES



Module 1 – Student Management

Responsible for creating and managing student profiles.

Module 2 – Free-Time Management

Responsible for storing and modifying student availability.

Module 3 – Group Management

Responsible for creating groups and managing members.

Module 4 – Common-Time Matching

Responsible for calculating overlapping free-time slots.

Module 5 – Activity Management

Responsible for creating and storing activities.

Module 6 – Conflict Detection

Responsible for checking whether a proposed activity overlaps another activity.

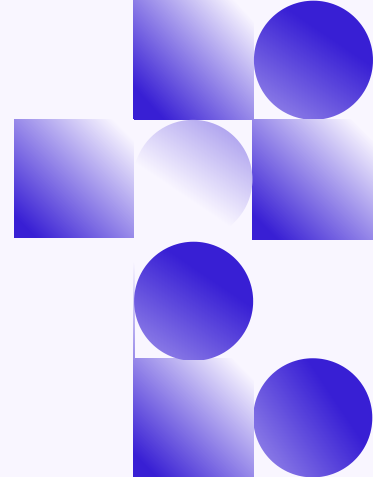
Module 7 – Reminder Management

Responsible for displaying upcoming activities.

Module 8 – Reporting

Responsible for generating activity statistics and summaries.

PROPOSED CLASS STRUCTURE



src/

|

|—— Main.java

|

|—— model/

|

|—— Student.java

|

|—— TimeSlot.java

|

|—— Group.java

|

|—— Activity.java

|

|—— Meeting.java

|

|—— service/

|

|—— StudentService.java

|

|—— ScheduleService.java

|

|—— MatchingService.java

|

|—— MeetingService.java

|

|—— ReportService.java

|

|—— exception/

| └── ScheduleException.java

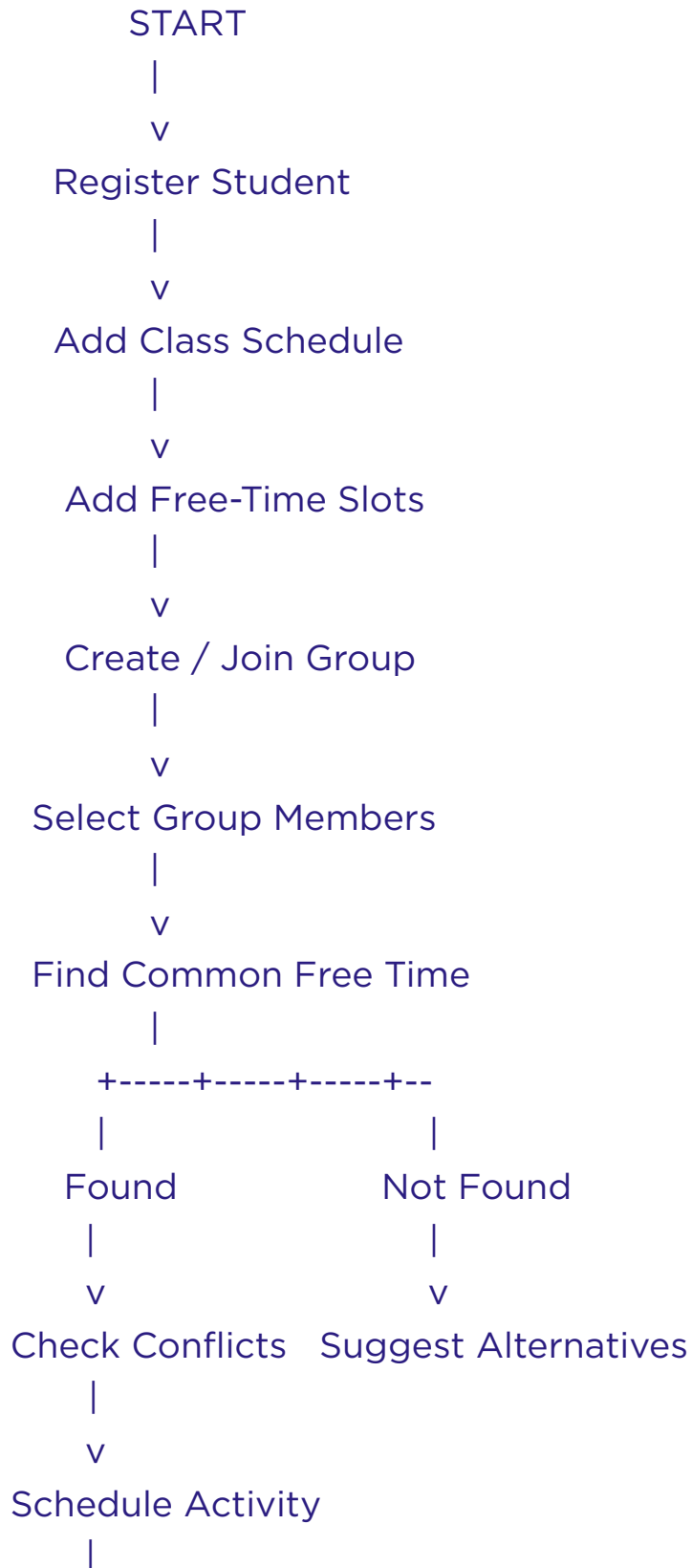
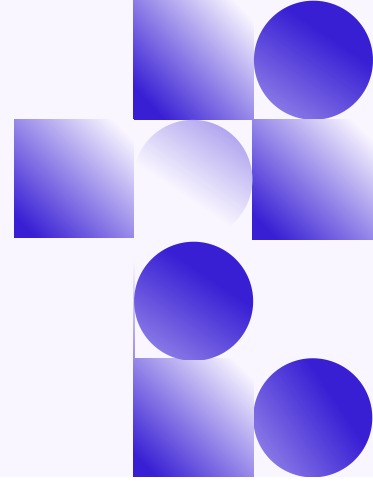
|

└── util/

 └── FileManager.java

 └── ValidationUtil.java

WORKFLOW



v

View Upcoming Activities

|

v

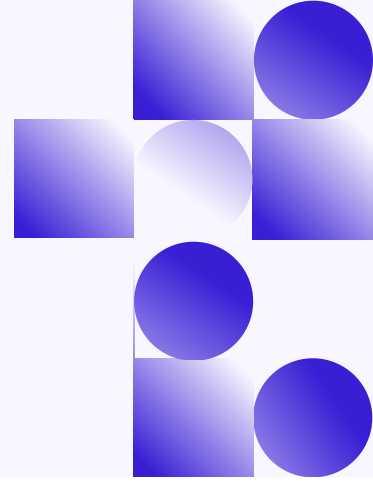
Generate Report

|

v

END

COMMON FREE TIME ALGORITHM



The system compares the free-time intervals of group members.

For two time slots:

Student A: 4:00 PM - 7:00 PM

Student B: 5:00 PM - 8:00 PM

The intersection is:

Start = maximum(4:00, 5:00)
= 5:00 PM

End = minimum(7:00, 8:00)
= 7:00 PM

Therefore:

Common Time = 5:00 PM - 7:00 PM

For multiple students, the process is repeated for every member.

If:

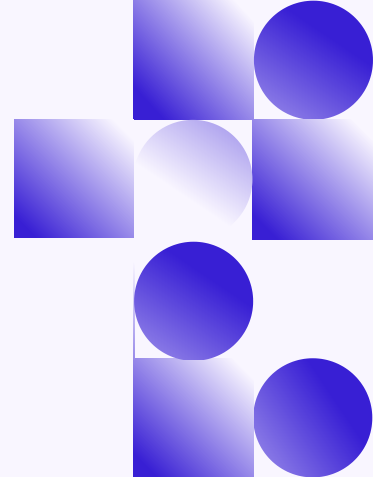
$\text{commonStart} < \text{commonEnd}$

then a common time exists.

Otherwise:

No common free time available.

CONFLICT DETECTION LOGIC



Two activities conflict if their time periods overlap.

For activities A and B:

$A.start < B.end$

AND

$B.start < A.end$

If both conditions are true, the activities overlap.

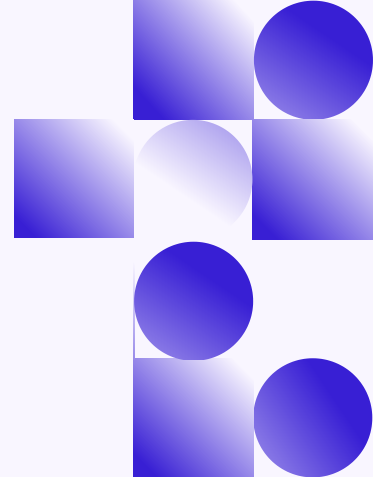
Example:

Existing Activity: 5:00 PM - 6:30 PM

New Activity: 6:00 PM - 7:00 PM

Since the periods overlap, the system rejects the new schedule.

EXCEPTION HANDLING



The project demonstrates Java exception-handling concepts.

The system can use:

- try
- catch
- multi-catch
- finally
- throw
- throws

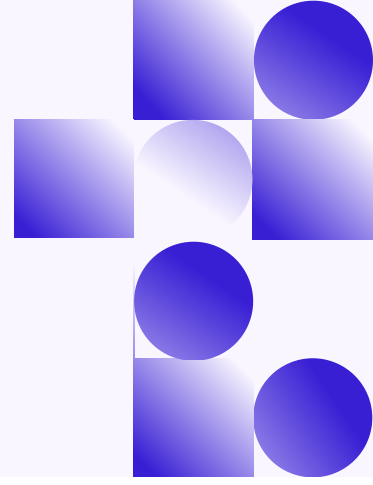
Example situations:

- Invalid student ID
- Invalid time format
- Missing file
- Invalid menu choice
- Duplicate student
- Scheduling conflict

A custom exception can be created:

```
public class ScheduleException extends Exception {  
    public ScheduleException(String message) {  
        super(message);  
    }  
}
```

ASSERTIONS



Assertions can be used to test internal invariants.

For example, a time slot should satisfy:

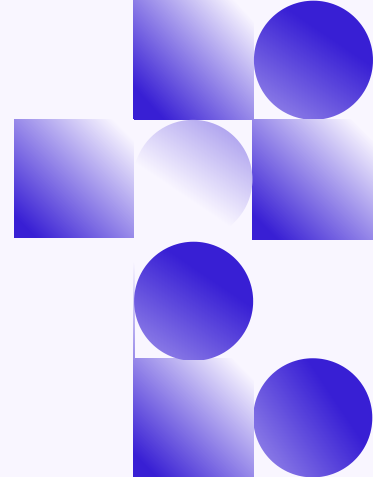
start time < end time

Example:

```
assert startTime.isBefore(endTime) :  
    "Start time must be before end time";
```

Assertions help detect programming errors during development and testing.

FILE HANDLING



The project uses Java's Files and Path classes for file operations.

Example data files:

```
data/
|—— students.txt
|—— schedules.txt
|—— groups.txt
|—— meetings.txt
```

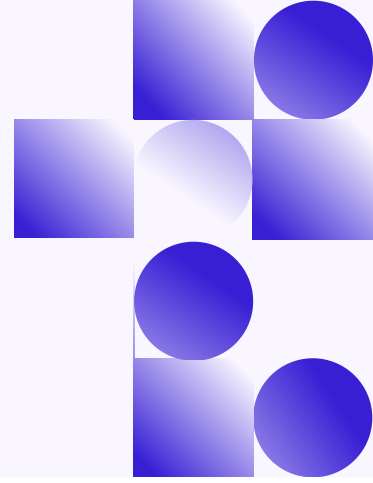
The Path interface can represent file and directory locations.

The Files class can be used to:

- Check whether a file exists.
- Create files.
- Read files.
- Write files.
- Copy files.
- Move files.
- Delete files.

This demonstrates practical Java file-handling concepts.

DATA DESIGN



Since the project uses file-based storage, the basic data structure can be represented as follows.

Student

Student

studentId

name

department

year

TimeSlot

TimeSlot

date

startTime

endTime

available

Group

Group

groupId

groupName

activityType

memberIds

Meeting

Meeting

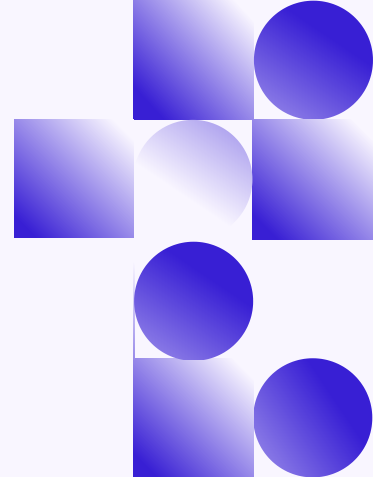
meetingId

groupId

...

title
date
startTime
endTime
location

USE CASE DIAGRAM DESCRIPTION



Actors

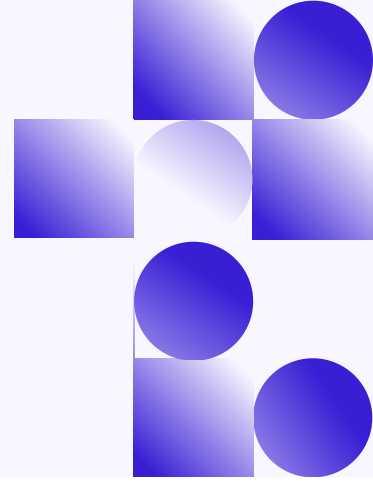
Student

Main Use Cases

Student

- |
- +---- Register
- |
- +---- Manage Profile
- |
- +---- Manage Schedule
- |
- +---- Manage Free Time
- |
- +---- Create Group
- |
- +---- Join Group
- |
- +---- Find Common Free Time
- |
- +---- Schedule Activity
- |
- +---- Check Conflict
- |
- +---- View Activities
- |
- +---- View Reports

CLASS DIAGRAM DESCRIPTION



The main relationships are:

Student

|

| has

v

TimeSlot

Group

|

| contains

v

Student

Group

|

| schedules

v

Meeting

Meeting

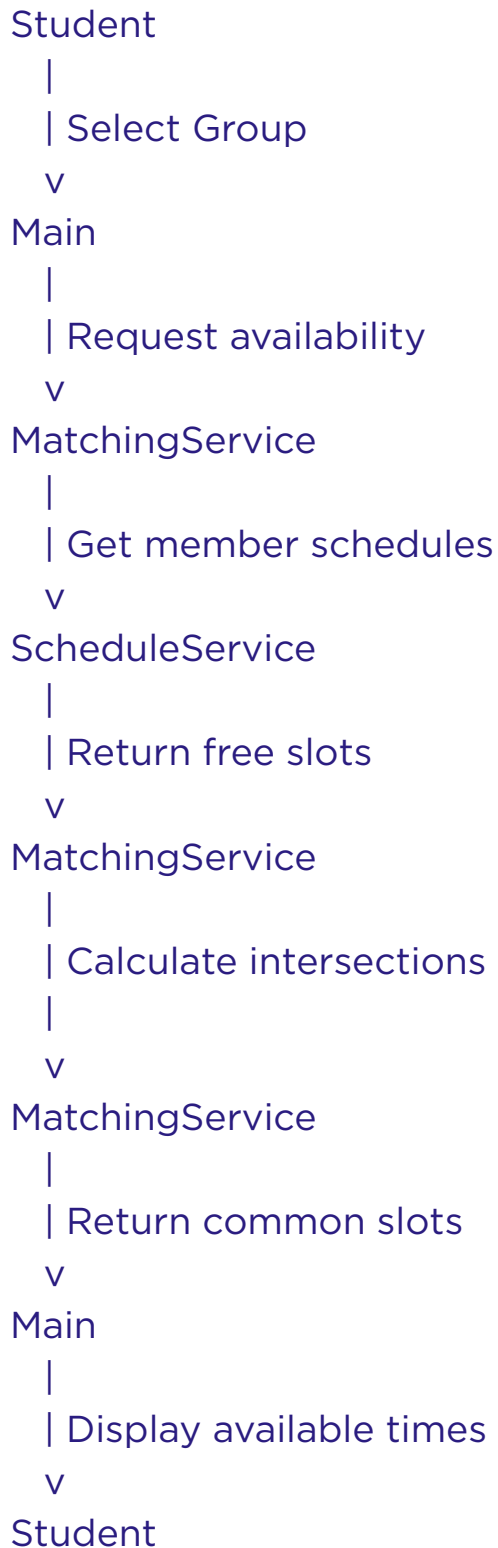
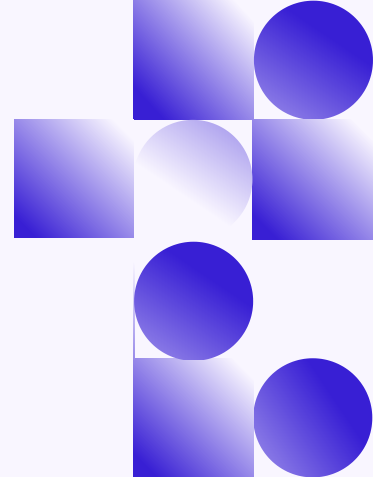
|

| represents

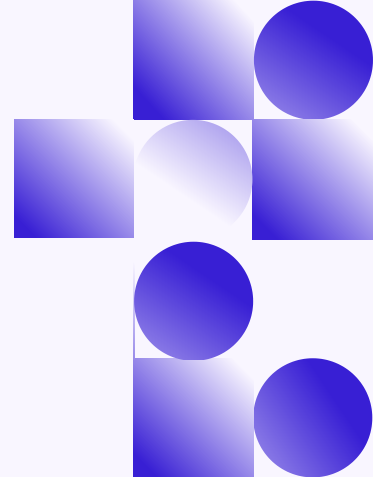
v

Activity

SEQUENCE DIAGRAM - FINDING COMMON FREE TIME



SAMPLE USER INTERFACE & IMPLEMENTATION



MAIN MENU

```
Explorer
└─ CSE2006_PROGRAMMING_IN_JAVA [Codespaces: effective fortnight]
   ├── data
   ├── out
   └── src
      ├── exception
      ├── model
      │   ├── Activity.java
      │   ├── Group.java
      │   ├── Student.java
      │   └── TimeSlot.java
      ├── service
      │   ├── ActivityService.java
      │   ├── MatchingService.java
      │   ├── MeetingService.java
      │   └── ScheduleService.java
      ├── util
      │   ├── Main.java
      │   ├── readme.md
      │   └── statement.md
      └── Outline
      └── Timeline

Terminal
@tanishasethi27 → /workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main
Directory already exists: data

=====
STUDENT FREE TIME MANAGEMENT SYSTEM
=====

MAIN MENU

1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit

Enter your choice: █
```

REGISTER A STUDENT

```
Terminal
@tanishasethi27 → /workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

=====
STUDENT FREE TIME MANAGEMENT SYSTEM
=====

MAIN MENU

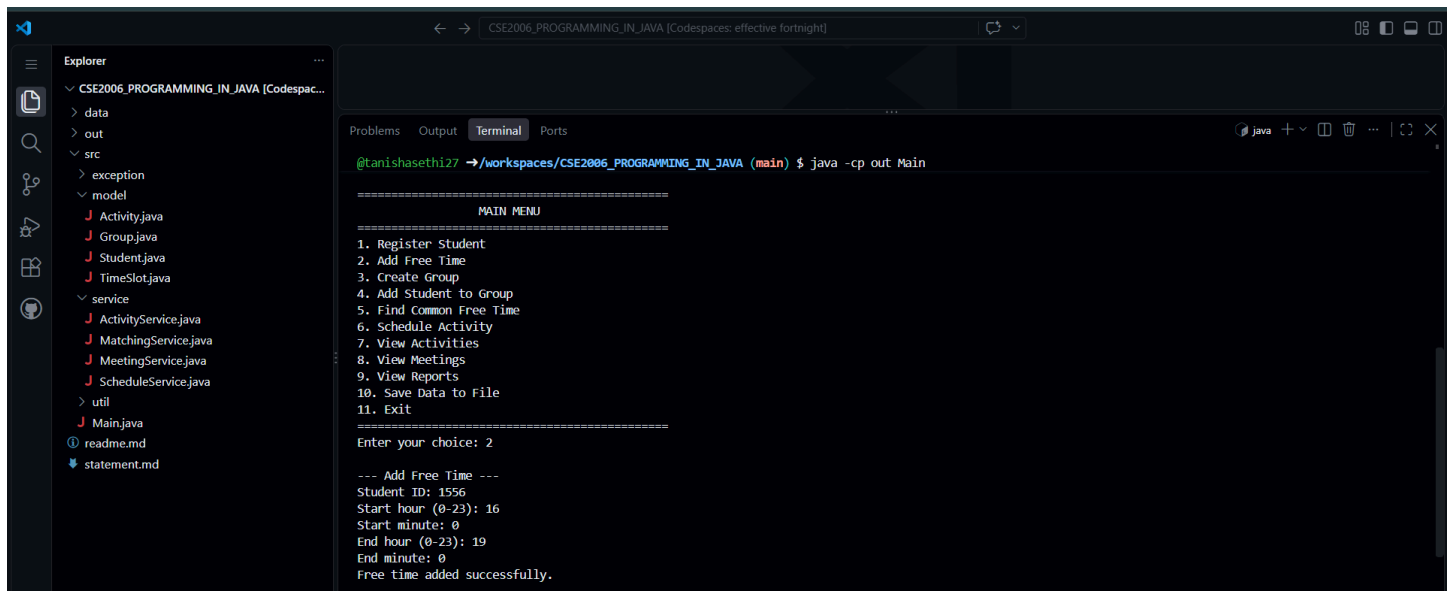
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit

Enter your choice: 1

--- Register Student ---
Student ID: 1556
Name: Priya Gupta
Department: CSE
Year: 2
Student registered successfully.

=====
```

ADDING FREE TIME



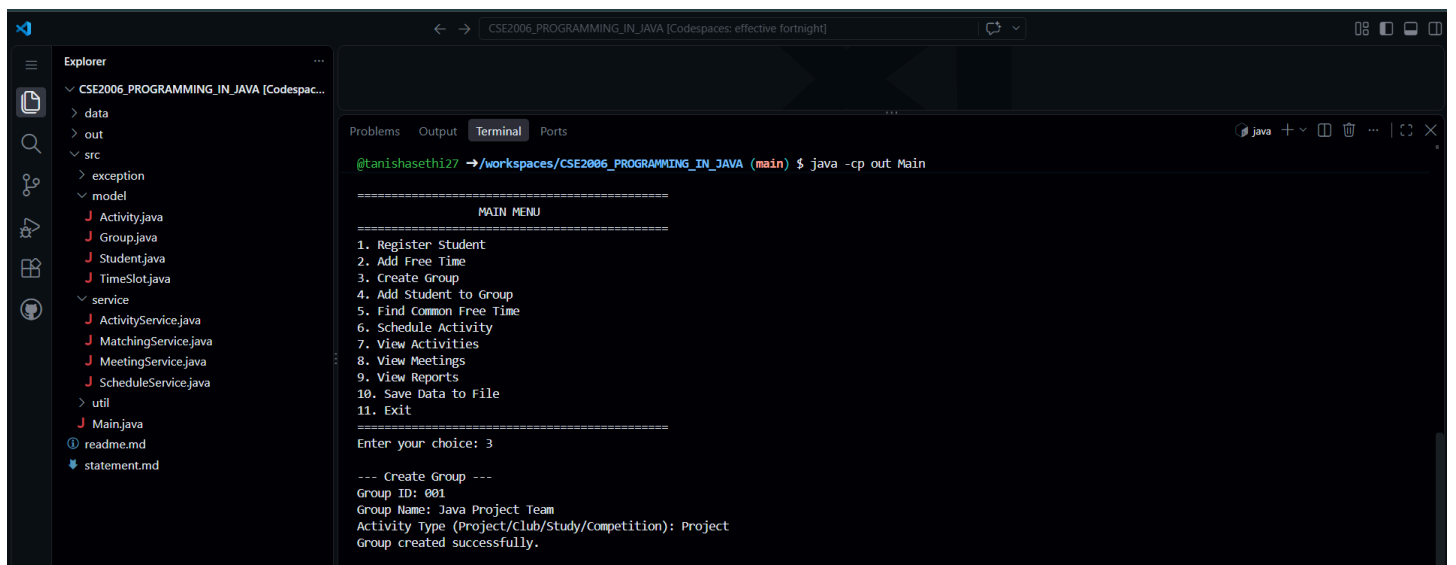
```
@tanishasethi27 →/workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

=====
MAIN MENU
=====
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit
=====

Enter your choice: 2

--- Add Free Time ---
Student ID: 1556
Start hour (0-23): 16
Start minute: 0
End hour (0-23): 19
End minute: 0
Free time added successfully.
```

CREATING A GROUP



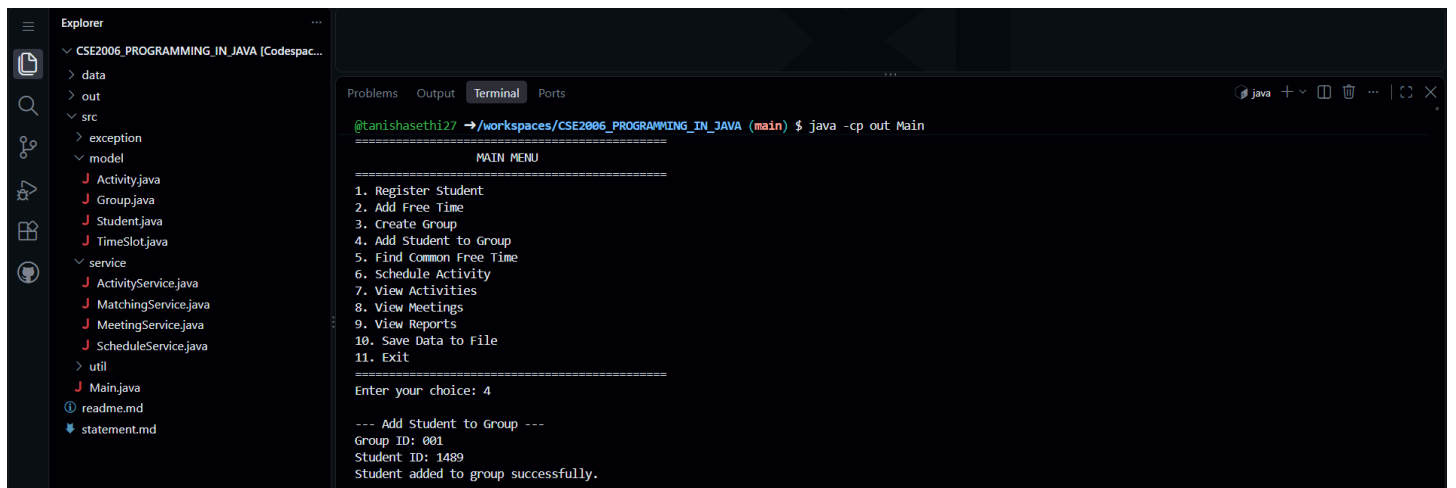
```
@tanishasethi27 →/workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

=====
MAIN MENU
=====
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit
=====

Enter your choice: 3

--- Create Group ---
Group ID: 001
Group Name: Java Project Team
Activity Type (Project/Club/Study/Competition): Project
Group created successfully.
```

ADDING STUDENT TO GROUP



```
@tanishasethi27 →/workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

=====
MAIN MENU
=====
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit
=====

Enter your choice: 4

--- Add Student to Group ---
Group ID: 001
Student ID: 1489
Student added to group successfully.
```

FINDING COMMON FREE TIME


```
@tanishasethi27 →/workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

=====
MAIN MENU
=====
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit
=====
Enter your choice: 5

--- Find Common Free Time ---
Group ID: 001

=====
COMMON FREE TIME
=====
17:30 - 19:30
```

SCHEDULING ACTIVITY

```
@tanishasethi27 →/workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

=====
MAIN MENU
=====
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit
=====
Enter your choice: 6

--- Schedule Activity ---
Activity ID: A1
Title: Java Project Discussion
Type (Project/Club/Study/Competition): Project
Date (YYYY-MM-DD): 2026-09-20
Start hour: 18
Start minute: 0
End hour: 19
End minute: 0
Location: Computer Lab
Activity added successfully.
Meeting scheduled successfully.
```

VIEWING SCHEDULED ACTIVITIES

```
@tanishasethi27 →/workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

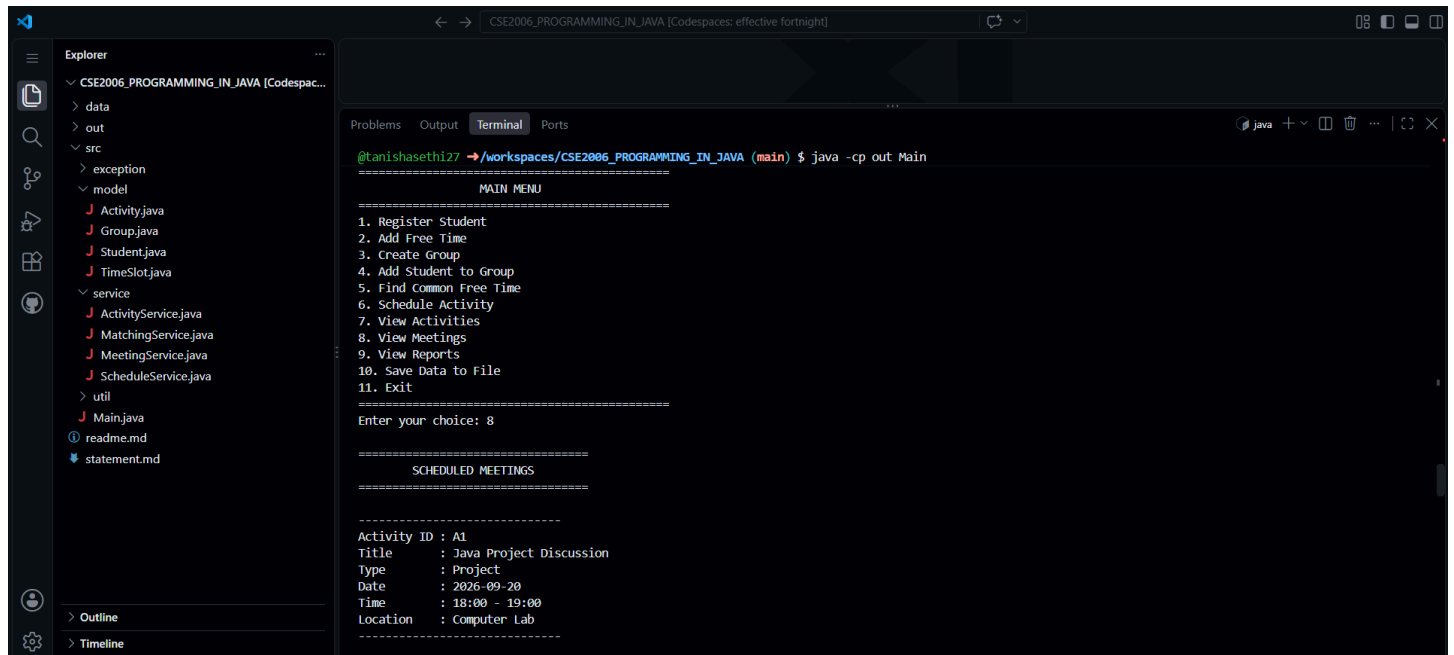
=====
MAIN MENU
=====
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit
=====
Enter your choice: 7

=====
SCHEDULED ACTIVITIES
=====

Activity ID : A1
Title      : Java Project Discussion
Type       : Project
Date       : 2026-09-20
Time       : 18:00 - 19:00
Location   : Computer Lab
```

VIEWING SCHEDULED MEETING

VIEWING SCHEDULED MEETING



The screenshot shows the VS Code interface with the Explorer on the left and the Terminal on the right. The Explorer shows a project structure for 'CSE2006_PROGRAMMING_IN_JAVA'. The Terminal shows the output of a Java application. The application has a 'MAIN MENU' with 11 options. Option 8, 'View Meetings', has been selected. The output displays the details of a scheduled meeting.

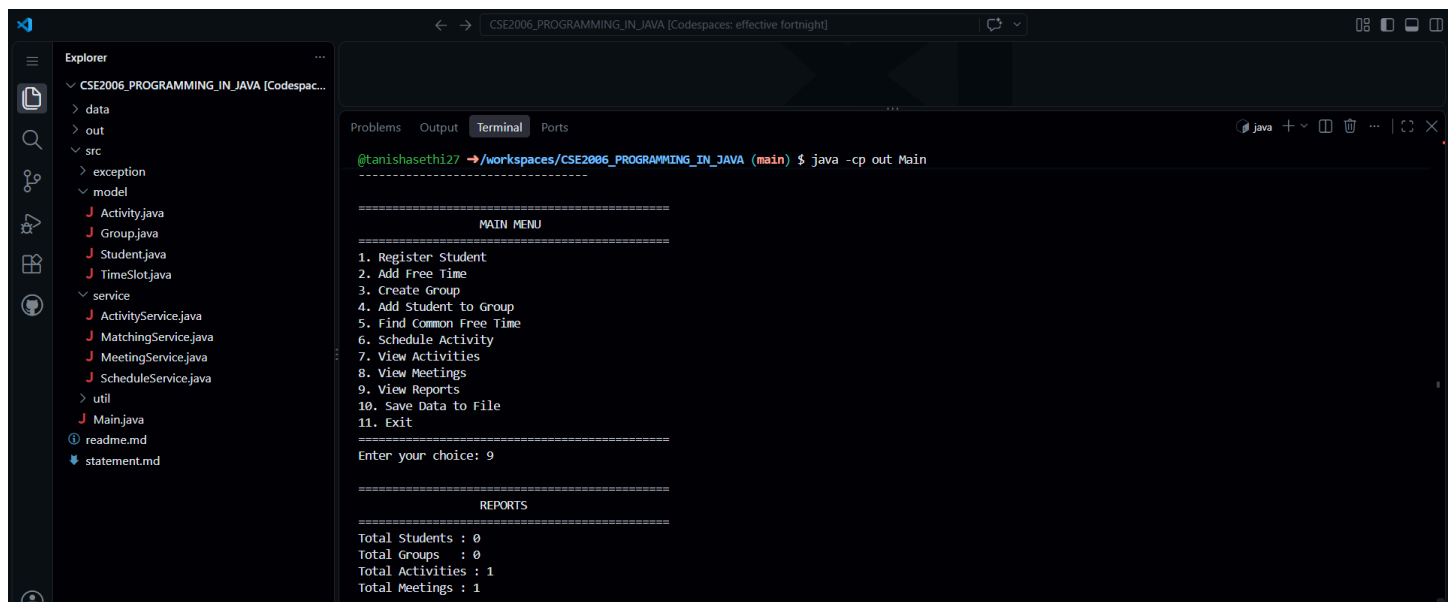
```
@tanishasethi27 →/workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

=====
MAIN MENU
=====
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit
=====
Enter your choice: 8

=====
SCHEDULED MEETINGS
=====

Activity ID : A1
Title      : Java Project Discussion
Type       : Project
Date       : 2026-09-20
Time       : 18:00 - 19:00
Location   : Computer Lab
```

VIEWING REPORTS



The screenshot shows the VS Code interface with the Explorer on the left and the Terminal on the right. The Explorer shows the same project structure. The Terminal shows the output of the Java application. The application has a 'MAIN MENU' with 11 options. Option 9, 'View Reports', has been selected. The output displays the total counts for students, groups, activities, and meetings.

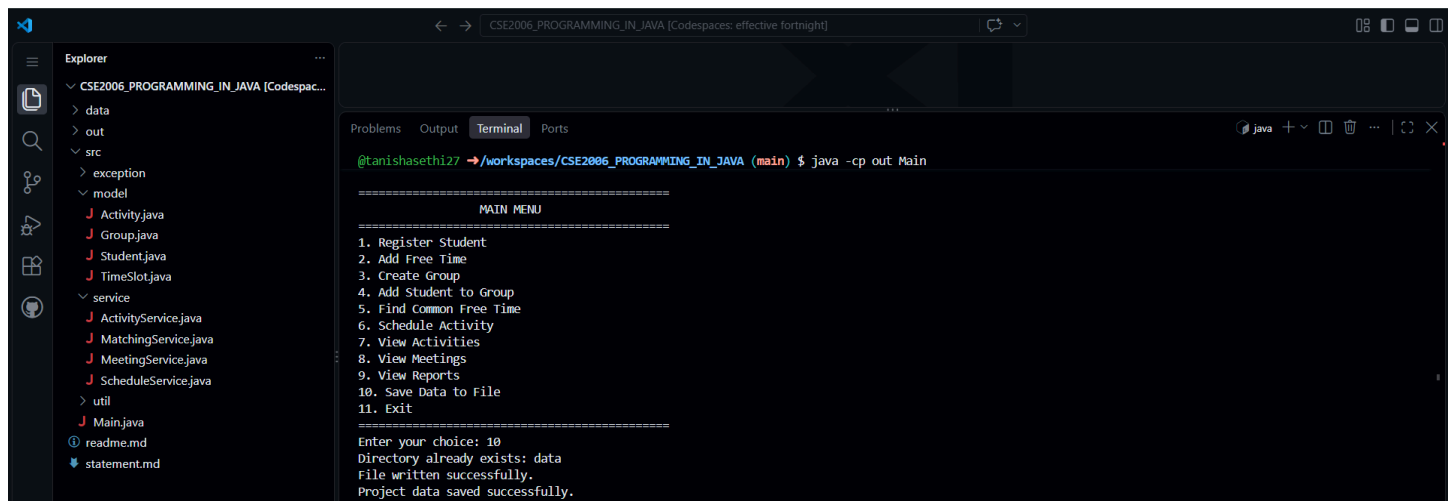
```
@tanishasethi27 →/workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

=====
MAIN MENU
=====
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit
=====
Enter your choice: 9

=====
REPORTS
=====

Total Students : 0
Total Groups   : 0
Total Activities : 1
Total Meetings : 1
```

SAVING DATA TO FILE

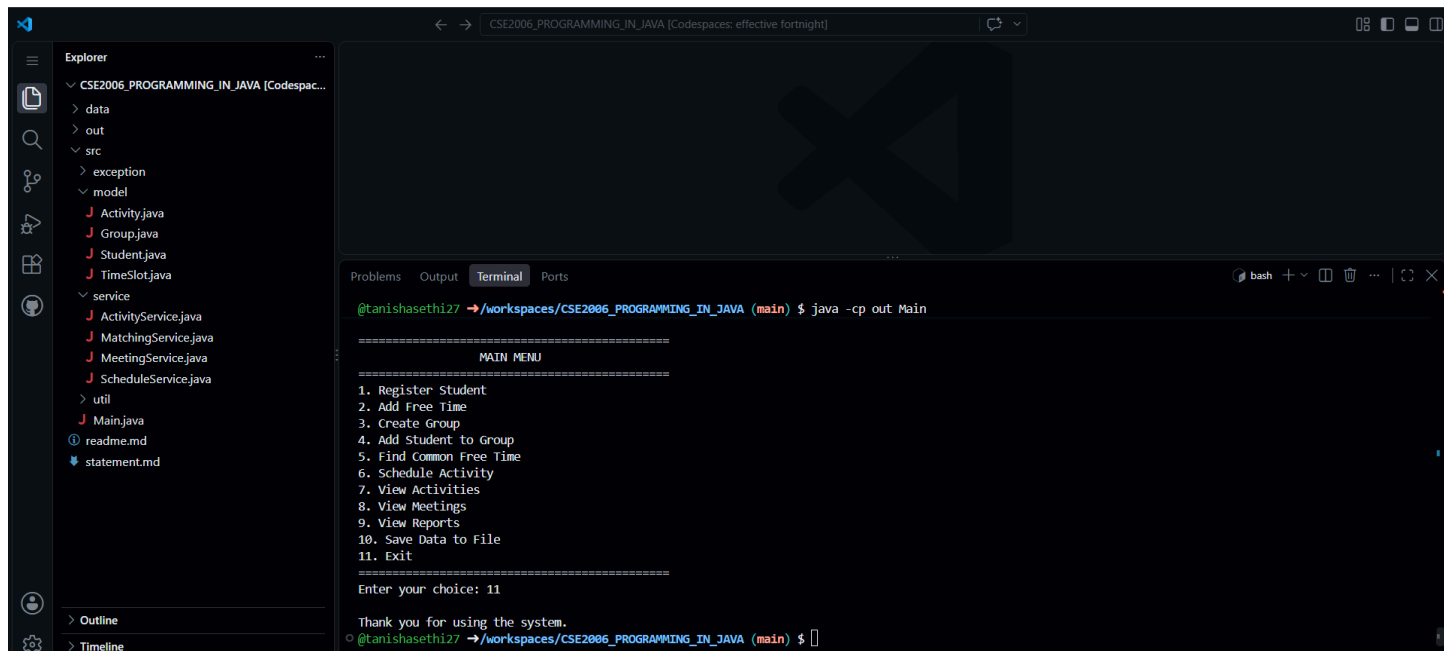


The screenshot shows the VS Code interface with the Explorer on the left and the Terminal on the right. The Explorer shows the same project structure. The Terminal shows the output of the Java application. The application has a 'MAIN MENU' with 11 options. Option 10, 'Save Data to File', has been selected. The output displays a confirmation message that the data has been saved successfully.

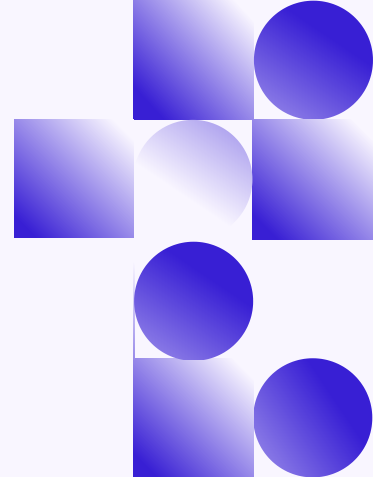
```
@tanishasethi27 →/workspaces/CSE2006_PROGRAMMING_IN_JAVA (main) $ java -cp out Main

=====
MAIN MENU
=====
1. Register Student
2. Add Free Time
3. Create Group
4. Add Student to Group
5. Find Common Free Time
6. Schedule Activity
7. View Activities
8. View Meetings
9. View Reports
10. Save Data to File
11. Exit
=====
Enter your choice: 10
Directory already exists: data
File written successfully.
Project data saved successfully.
```

EXIT



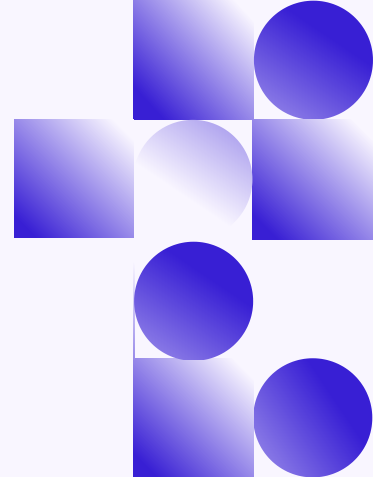
TESTING STRATEGY



The project will be tested using different test cases.

Test Case	Input	Expected Result
Register student	Valid details	Student registered
Register student	Duplicate ID	Error message
Add free time	Valid time	Slot added
Add free time	End before start	Validation error
Find common time	Overlapping slots	Common slot displayed
Find common time	No overlap	No common time message
Schedule activity	Available time	Activity scheduled
Schedule activity	Conflicting time	Conflict error
Read file	Existing file	Data loaded
Read file	Missing file	Proper error handling
Invalid menu	99	Invalid choice message

ERROR HANDLING



The system should handle errors gracefully.

Example:

Invalid input.

Please enter a valid time in HH:MM format.

For a scheduling conflict:

Scheduling failed!

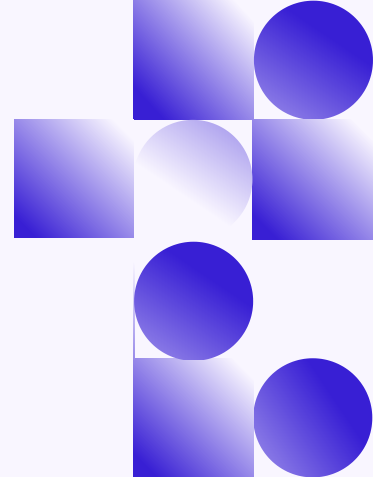
Reason:

The selected time overlaps with an existing activity.

Please select another available time.

The application should continue running instead of terminating unexpectedly.

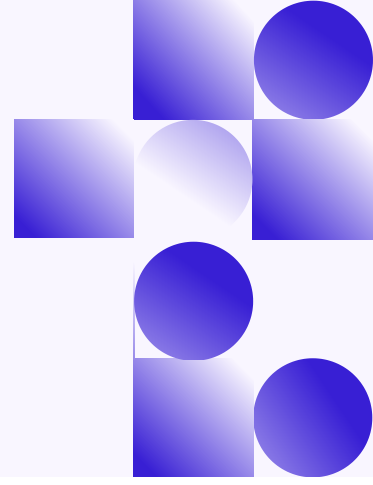
ADVANTAGES



The proposed system provides several advantages:

- Reduces manual schedule coordination.
- Quickly identifies common availability.
- Reduces scheduling conflicts.
- Centralizes group activity information.
- Easy to operate through a terminal.
- Demonstrates multiple Java concepts.
- Can be extended with a database or GUI in the future.
- Provides a practical solution to a common student problem.

CHALLENGES

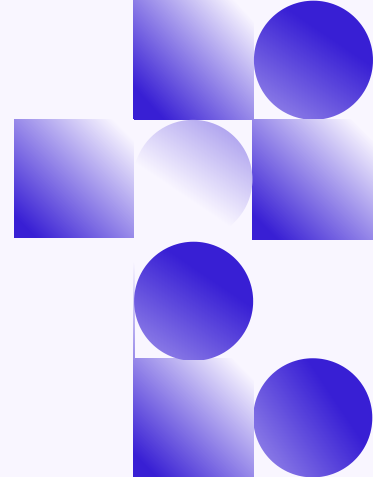


Potential challenges during implementation include:

1. Handling multiple students' schedules.
2. Correctly calculating overlapping time intervals.
3. Validating time and date input.
4. Detecting conflicting meetings.
5. Managing file operations.
6. Handling invalid user input.
7. Maintaining clean separation between different modules.

These challenges provide opportunities to apply Java programming concepts in a practical project.

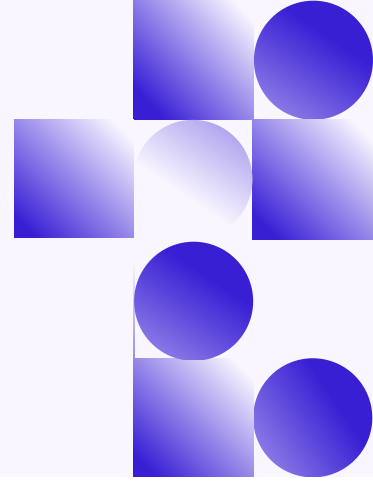
LEARNING OUTCOMES



After completing this project, the following concepts will be demonstrated:

- Object-oriented programming
- Classes and objects
- Encapsulation
- Collections
- ArrayList
- HashMap
- HashSet
- Exception handling
- Custom exceptions
- Assertions
- File handling
- Files class
- Path interface
- Input validation
- Algorithms
- Modular programming
- Git and GitHub
- Software design
- Testing

FUTURE ENHANCEMENTS



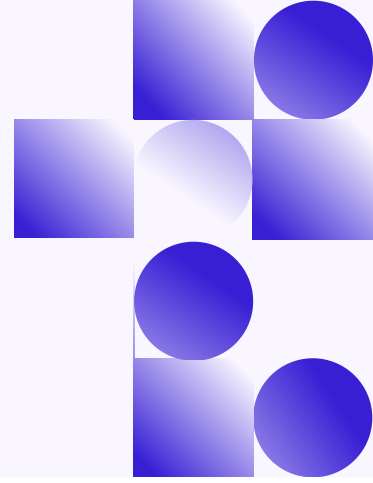
The project can be improved in the future by adding:

1. MySQL or another database.
2. Web-based interface.
3. Android/mobile application.
4. Login and authentication.
5. Email notifications.
6. Push notifications.
7. Calendar integration.
8. Automatic meeting recommendations.
9. Cloud synchronization.
10. AI-based schedule recommendations.

For example, the future system could recommend:

"The best time for all four members is Wednesday, 6:00 PM-7:00 PM."

CONCLUSION

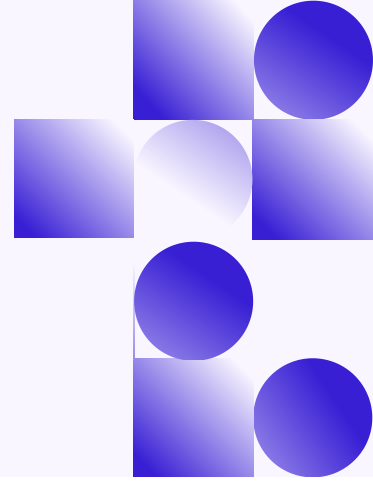


The **SyncSpace** provides a practical solution for coordinating student activities. The system allows students to manage their availability, create groups, find common free time, schedule activities, and detect conflicts.

The project also demonstrates important Java programming concepts including object-oriented programming, collections, exception handling, assertions, file handling, Files, and Path.

The modular architecture makes the application easier to maintain and extend. Future versions can incorporate databases, graphical interfaces, mobile applications, notifications, and intelligent scheduling.

REFERENCES



1. [Java SE Documentation - Oracle](#)
2. [Java Collections Framework Documentation](#)
3. [Java Files API Documentation](#)
4. [Java Path API Documentation](#)
5. [Git Documentation](#)
6. [GitHub Documentation](#)